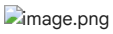


```
import pandas as pd
```

```
df=pd.read_csv('weatherAUS.csv', encoding='utf-8', usecols=['Date', 'Location', 'MinTemp', 'MaxTemp'])
```

```
#Drop records where target MinTemp=NaN or MaxTemp=NaN
df = df.dropna(subset=['MinTemp', 'MaxTemp'])
```

```
# Convert dates to year-months
df['Year-Month']= (pd.to_datetime(df['Date'], yearfirst=True)).dt.strftime('%Y-%m')
```

```
# Derive median daily temperature (mid point between Daily Max and Daily Min)
df['MedTemp']=df[['MinTemp', 'MaxTemp']].median(axis=1)
```

```
df
```

	Date	Location	MinTemp	MaxTemp	Year-Month	MedTemp
0	2008-12-01	Albury	13.4	22.9	2008-12	18.15
1	2008-12-02	Albury	7.4	25.1	2008-12	16.25
2	2008-12-03	Albury	12.9	25.7	2008-12	19.30
3	2008-12-04	Albury	9.2	28.0	2008-12	18.60
4	2008-12-05	Albury	17.5	32.3	2008-12	24.90
...	...	...	...	...	...	...
65293	2012-02-07	MelbourneAirport	10.9	21.2	2012-02	16.05
65294	2012-02-08	MelbourneAirport	8.9	21.7	2012-02	15.30
65295	2012-02-09	MelbourneAirport	9.2	26.0	2012-02	17.60
65296	2012-02-10	MelbourneAirport	15.3	25.8	2012-02	20.55
65297	2012-02-11	MelbourneAirport	14.5	20.9	2012-02	17.70

64463 rows × 6 columns

Aggregate to monthly averages

```
# Create a copy of an original dataframe
df2=df[['Location', 'Year-Month', 'MedTemp']].copy()
```

```
# Calculate monthly average temperature for each location
df2 = df2.groupby(['Location', 'Year-Month'], as_index=False)['MedTemp'].mean()
```

```
# Transpose dataframe
# Pivot to wide format
df2_pivot = df2.pivot(index='Location', columns='Year-Month', values='MedTemp')
```

```
df2_pivot
```

Year-Month Location	2007-11	2007-12	2008-01	2008-02	2008-03	2008-04	2008-05	2008-06	2008-07	2008-08	...	2016-09	2016-10	201
Albury	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	12.213333	13.617742	18.17
BadgerysCreek	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	14.766667	17.037097	21.48
Ballarat	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	9.426667	10.600000	13.42
Bendigo	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	11.011667	12.595161	16.81
Canberra	18.403333	19.216129	22.237097	18.636207	18.427419	12.341667	9.885484	9.475000	5.803226	6.585484	...	10.295000	11.838710	16.70
Cobar	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	13.121667	17.475806	22.84
CoffsHarbour	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	17.513793	18.382258	21.33
MelbourneAirport	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	
Moree	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	15.043333	18.154839	23.23
MountGinini	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	3.748333	4.767742	10.35
Newcastle	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	16.526923	18.301786	21.66
NorahHead	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	17.098333	18.709677	21.35
NorfolkIsland	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	17.730000	17.996774	19.42
Penrith	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	16.121667	18.448387	22.66
Richmond	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	15.438333	17.800000	22.15
Sale	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	12.321667	13.587097	15.62
Sydney	NaN	NaN	NaN	21.627586	21.216129	17.711667	15.967742	15.258333	12.751613	12.733871	...	17.215000	19.108065	21.73
SydneyAirport	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	17.228333	19.008065	21.73
Tuggeranong	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	10.311667	12.256452	17.07
WaggaWagga	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	11.848333	13.580645	18.51
Williamtown	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	16.310000	18.116129	21.15
Wollongong	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	16.216667	17.701613	19.90

22 rows × 113 columns

```
# Remove locations with excessive missing values (NaN)
df2_pivot = df2_pivot.drop(
    ['Dartmoor', 'Katherine', 'Melbourne', 'Nhil', 'Uluru'],
    axis=0, errors='ignore'
)
```

```
# Remove months with lots of missing data (NaN)
df2_pivot=df2_pivot.drop(['2007-11', '2007-12', '2008-01', '2008-02', '2008-03', '2008-04', '2008-05', '2008-06', '2008-07', '2008-08', '2008-09', '200
```

```
df2_pivot
```

Year-Month Location	2009-01	2009-02	2009-03	2009-04	2009-05	2009-06	2009-07	2009-08	2009-09	2009-10	...	2016-03	2016-04	201
Albury	25.485484	25.439286	20.535484	15.410000	11.716129	9.206667	8.201613	9.932258	11.951786	14.595161	...	23.290323	17.580000	13.06
BadgerysCreek	24.467742	22.948214	20.877419	17.305000	14.312903	11.769643	10.727419	12.520968	15.558333	16.646429	...	22.938710	19.743333	15.16
Ballarat	19.498387	19.605357	16.440323	12.175000	10.222581	7.768333	7.108065	8.366129	9.555000	11.288710	...	18.667742	14.500000	11.00
Bendigo	22.995161	22.932143	18.941935	14.511667	12.095161	8.640000	7.837097	9.635484	10.696667	13.859677	...	22.119355	16.440000	12.32
Canberra	22.512903	21.523214	19.219355	14.030000	10.343548	8.023333	6.827419	8.267742	10.676667	12.614516	...	20.122581	16.068333	11.07
Cobar	29.264516	26.864286	24.611290	18.898333	14.756452	12.123333	10.833871	14.095161	16.546667	19.229032	...	25.532258	21.925000	15.65
CoffsHarbour	23.301613	23.274074	22.158065	19.828333	16.774194	14.343333	13.509677	16.058065	17.341667	19.064516	...	23.261290	21.310000	17.75
MelbourneAirport	21.116129	21.067857	18.587097	15.078333	12.335484	10.220000	9.854839	11.600000	12.831667	13.706452	...	NaN	NaN	
Moree	27.238710	25.946429	23.774194	19.895000	15.461290	12.940000	11.370968	15.030645	16.683333	20.220968	...	26.458065	22.911667	17.01
MountGinini	15.966129	14.298214	12.140323	6.480000	4.129032	1.116071	-0.541667	1.459259	4.635714	5.387500	...	13.495161	9.950000	4.84
Newcastle	24.430645	23.792857	22.219355	18.813889	16.335000	13.882143	12.600000	14.890323	16.965789	18.542593	...	22.542105	20.538000	17.56
NorahHead	23.090000	22.085714	22.298387	19.343333	17.112903	15.213793	13.748387	15.951613	17.789655	18.159091	...	22.750000	21.046667	18.19
NorfolkIsland	21.545161	23.407143	22.179032	21.006667	17.711290	17.200000	16.035484	16.608065	17.193333	17.283333	...	23.103226	21.881667	20.10
Penrith	25.791935	24.030357	22.090323	18.240000	15.470968	12.641667	11.819355	13.676667	17.090000	18.187097	...	24.129032	20.841667	16.32
Richmond	24.277419	23.642857	21.401613	17.741667	14.390323	12.056667	10.991935	12.724194	16.230000	17.146774	...	23.106667	19.900000	15.30
Sale	19.685484	19.258929	17.806452	14.006667	11.103226	9.080000	9.262903	10.909677	12.233333	13.161290	...	19.266129	15.665000	12.90
Sydney	23.729032	22.708929	22.156452	19.245000	16.601613	14.065000	13.751613	15.617742	18.265000	17.746774	...	23.214516	20.805000	18.06
SydneyAirport	24.146774	22.867857	22.474194	19.113333	16.403226	14.198333	13.383871	15.708065	18.211667	17.979032	...	23.314516	20.805000	18.06
Tuggeranong	22.701613	21.887500	19.135484	14.090000	10.329032	7.900000	6.838710	8.145161	10.721667	12.816129	...	20.070968	15.828333	10.67
WaggaWagga	26.177419	25.891071	21.500000	16.218333	12.880645	9.943333	8.435484	9.806452	12.265000	15.398387	...	23.490323	18.420000	12.95
Williamtown	23.777419	23.385714	21.206452	18.585000	15.770968	13.156667	11.982258	14.172581	16.816667	17.172581	...	22.882258	19.918333	16.76
Wollongong	22.146774	21.162500	21.162903	18.206667	16.541379	14.633333	13.641935	15.593548	17.135000	16.890323	...	21.464516	20.120000	17.78

22 rows × 93 columns

```
# Remove months with insufficient data
months_to_drop = [
    '2007-11','2007-12',
    '2008-01','2008-02','2008-03','2008-04','2008-05','2008-06',
    '2008-07','2008-08','2008-09','2008-10','2008-11','2008-12',
    '2017-01','2017-02','2017-03','2017-04','2017-05','2017-06'
]
df2_pivot = df2_pivot.drop(months_to_drop, axis=1, errors='ignore')
```

```
# Add missing months 2011-04, 2011-04, 2011-04 and impute data
df2_pivot['2011-04']=(df2_pivot['2011-03']+df2_pivot['2011-05'])/2
df2_pivot['2012-12']=(df2_pivot['2012-11']+df2_pivot['2013-01'])/2
df2_pivot['2013-02']=(df2_pivot['2013-01']+df2_pivot['2013-03'])/2
```

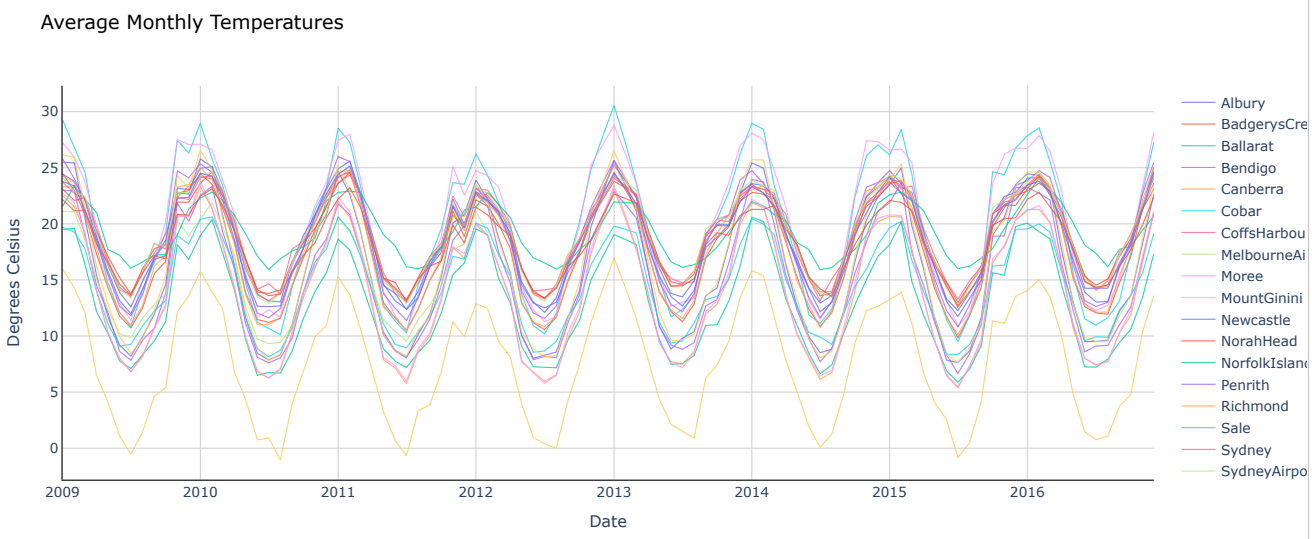
```
# Sort columns so Year-Months are in the correct order
df2_pivot=df2_pivot.reindex(sorted(df2_pivot.columns), axis=1)
```

```
import plotly.graph_objects as go
```

```
# Plot average monthly temperature derived from daily medians for each location
fig = go.Figure()
for location in df2_pivot.index:
    fig.add_trace(go.Scatter(x=df2_pivot.loc[location, :].index,
                             y=df2_pivot.loc[location, :].values,
                             mode='lines', name=location,
                             opacity=0.8, line=dict(width=1)
                            ))
```

```
# Change chart background color
fig.update_layout(dict(plot_bgcolor = 'white'), showlegend=True)

# Update axes lines
fig.update_xaxes(showgrid=True, gridwidth=1, gridcolor='lightgrey',
                 zeroline=True, zerolinewidth=1, zerolinecolor='lightgrey',
                 showline=True, linewidth=1, linecolor='black',
                 title='Date'
                )

fig.update_yaxes(showgrid=True, gridwidth=1, gridcolor='lightgrey',
                 zeroline=True, zerolinewidth=1, zerolinecolor='lightgrey',
                 showline=True, linewidth=1, linecolor='black',
                 title='Degrees Celsius'
```

```
                )

# Set figure title
fig.update_layout(title=dict(text="Average Monthly Temperatures", font=dict(color='black')))

fig.show()
```




```
import numpy as np
from sklearn.preprocessing import MinMaxScaler # for feature scaling
```

```
def shaping(datain, timestep, scaler):

    # Convert input dataframe to array and flatten
    arr = datain.to_numpy().flatten()

    # Scale
    arr=scaler.fit_transform(arr.reshape(-1, 1)).flatten()

    X_list, y_list = [], []
    for i in range(len(datain.columns) - 2 * timestep + 1):
        X_list.append(arr[i:i+timestep])
        y_list.append(arr[i+timestep:i+2*timestep])

    # Reshape input and target arrays
    X_out = np.array(X_list).reshape(-1, timestep, 1)
    y_out = np.array(y_list).reshape(-1, timestep, 1)
    return X_out, y_out
```

```
##### Step 1 - Specify parameters
timestep=18
location='Canberra'
scaler = MinMaxScaler(feature_range=(-1, 1))
```

```
##### Step 2 - Prepare data

# Split data into train and test dataframes
df_train=df2_pivot.iloc[:, :-2*timestep].copy()
df_test=df2_pivot.iloc[:, -2*timestep:].copy()
```

```
# Select one location
dfloc_train = df_train[df_train.index==location].copy()
dfloc_test = df_test[df_test.index==location].copy()
```

```
# Use previously defined shaping function to reshape the data for LSTM
X_train, y_train = shaping(datain=dfloc_train, timestep=timestep, scaler=scaler)
X_test, y_test = shaping(datain=dfloc_test, timestep=timestep, scaler=scaler)
```

```
from tensorflow import keras # for building Neural Networks
from keras.models import Sequential # for creating a linear stack of layers for our Neural Network
from keras import Input # for instantiating a keras tensor
from keras.layers import Bidirectional, GRU, RepeatVector, Dense, TimeDistributed # for creating layers inside the Neural Network
```

```
##### Step 3 - Specify the structure of a Neural Network
model = Sequential(name="GRU-Model")
# Input Layer - need to speicfy the shape of inputs
model.add(Input(shape=(X_train.shape[1],X_train.shape[2]), name='Input-Layer'))
```

```
# Encoder Layer
model.add(Bidirectional(GRU(units=32, activation='tanh', recurrent_activation='sigmoid', stateful=False), name='Hidden-GRU-Encoder-Layer'))
# Repeat Vector
model.add(RepeatVector(y_train.shape[1], name='Repeat-Vector-Layer'))
# Decoder Layer
model.add(Bidirectional(GRU(units=32, activation='tanh', recurrent_activation='sigmoid', stateful=False, return_sequences=True), name='Hidden-GRU-Decod
# Output Layer, Linear(x) = x
model.add(TimeDistributed(Dense(units=1, activation='linear'), name='Output-Layer'))
```

```
##### Step 4 - Compile the model
model.compile(
    optimizer='adam', loss='mean_squared_error',
    metrics=['mean_squared_error', 'mean_absolute_error']
)
```

```
history = model.fit(X_train, y_train,
    batch_size=1, epochs=100,
    validation_split=0.2, shuffle=True, verbose=0
)
```

```
##### Step 6 - Use model to make predictions
# Predict results on training data
pred_train = model.predict(X_train)
# Predict esults on test data
pred_test = model.predict(X_test)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 944ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 914ms/step
```

```
##### Step 7 - Print Performance Summary
print("")
print('-------------------- Model Summary --------------------')
model.summary() # print model summary
print("")
print('-------------------- Weights and Biases --------------------')
print("Too many parameters to print but you can use the code provided if needed")
print("")
for layer in model.layers:
    print(layer.name)
    for item in layer.get_weights():
        print("  ", item)
print("")
```

Show hidden output

```
# Print the last value in the evaluation metrics contained within history file
print('-------------------- Evaluation on Training Data --------------------')
for item in history.history:
    print("Final", item, ":", history.history[item][-1])
print("")
```

```
-------------------- Evaluation on Training Data --------------------
Final loss : 0.009551760740578175
Final mean_absolute_error : 0.07848216593265533
Final mean_squared_error : 0.009551760740578175
Final val_loss : 0.03644381836056709
Final val_mean_absolute_error : 0.161009281873703
Final val_mean_squared_error : 0.03644381836056709
```

```
# Evaluate the model on the test data using "evaluate"
print('-------------------- Evaluation on Test Data --------------------')
results = model.evaluate(X_test, y_test)
print("")
```

```
-------------------- Evaluation on Test Data --------------------
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step - loss: 0.0687 - mean_absolute_error: 0.2131 - mean_squared_error: 0.0687
```

```
# Plot average monthly temperatures (actual and predicted) for test (out of time) data
fig = go.Figure()

# Trace for actual temperatures
fig.add_trace(go.Scatter(x=np.array(dfloc_test.columns),
                         y=np.array(dfloc_test.values).flatten(),
                         mode='lines',
                         name='Average Monthly Temperatures - Actual (Test)',
                         opacity=0.8,
                         line=dict(color='black', width=1)
                        ))

# Trace for predicted temperatures
fig.add_trace(go.Scatter(x=np.array(dfloc_test.columns[-timestep:]),
                         y=scaler.inverse_transform(pred_test.reshape(-1,1)).flatten(),
                         mode='lines',
                         name='Average Monthly Temperatures - Predicted (Test)',
                         opacity=0.8
```

```
                         line=dict(color='red', width=1)
                        ))

# Change chart background color
fig.update_layout(dict(plot_bgcolor = 'white'))

# Update axes lines
fig.update_xaxes(showgrid=True, gridwidth=1, gridcolor='lightgrey',
                 zeroline=True, zerolinewidth=1, zerolinecolor='lightgrey',
                 showline=True, linewidth=1, linecolor='black',
                 title='Year-Month'
                )

fig.update_yaxes(showgrid=True, gridwidth=1, gridcolor='lightgrey',
                 zeroline=True, zerolinewidth=1, zerolinecolor='lightgrey',
                 showline=True, linewidth=1, linecolor='black',
                 title='Degrees Celsius'
```